



INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

ASSIGNMENT

Assignment Title: Firewall, Traffic Control and Forensic Verification

Student Name: Ahmad Zubairu Garba

Student ID: 2025/FWSD/11463

Course Title: Basic Networking Skills for Digital Forensics

Course Code: SBT-DF203

Instructor: Aminu Idris

September 19, 2026

EXECUTIVE SUMMARY

This report documents the execution and forensic findings of Lab 6: Firewall Traffic Control and Forensic Verification. The primary objective was to validate a Linux host-based firewall using iptables by capturing baseline HTTP traffic from an authorized lab client, applying a narrowly scoped blocking rule against that client, capturing the blocked connection behaviour, correlating packet-level evidence with firewall rule counters, and removing the rule to restore access all while preserving chain-of-custody integrity.

All practical activities were conducted inside an isolated Debian 12 virtual machine running on VMware Workstation, with Apache 2.4.68 serving a training page on TCP port 80. A secondary IP alias (`192.168.88.140`) was added to interface `eth0` to serve as the authorized lab client, while the machine's primary address (`192.168.88.131`) hosted the protected HTTP service. Analysis confirmed that the allowed baseline achieved a complete TCP three-way handshake and returned `HTTP/1.1 200 OK`, while the blocked session produced repeated SYN retransmissions with no SYN-ACK and a client-side connection timeout. SHA-256 cryptographic hashing verified byte-for-byte evidence integrity, and the `iptables` INPUT-chain packet/byte counters independently corroborated that the blocking rule matched the client's traffic.

SECTION A: THEORETICAL OVERVIEW & CORE CONCEPTS

1. Host-Based vs Network-Based Firewalls and the iptables Model

A firewall is any system that inspects, allows, or denies network traffic according to a defined policy. In digital forensics, understanding where a firewall sits in the traffic path is critical because it determines what evidence will and will not appear in a packet capture.

A network-based firewall is a dedicated appliance or virtual instance positioned between network segments. It filters traffic for many hosts simultaneously and is typically stateful. A host-based firewall, which is the focus of this lab, runs on the endpoint itself and filters traffic entering, leaving, or passing through that specific machine. Because it operates at the host, its logs and counters become local digital evidence that can be correlated with PCAP observations.

Linux implements host-based firewalling through the Netfilter framework, which is configured by the iptables user-space tool. Netfilter uses three built-in chains, each attached to a specific point in the packet's lifecycle:

1. INPUT: packets destined for the local host itself. This is where this lab's blocking rule was applied.
2. OUTPUT: packets generated by the local host and leaving it.
3. FORWARD: packets that arrive at the host but are not destined for it (i.e., the host is acting as a router). Not relevant to this lab.

Each chain evaluates a sequence of rules in order, and each rule defines a match condition (source IP, protocol, port, etc.) and a target action. The two most common targets are:

- ACCEPT: permit the packet to continue.
- DROP: silently discard the packet with no notification to the sender.
- REJECT: discard the packet *and* return an error to the sender (typically TCP RST or ICMP port-unreachable).

A modern interface to iptables is UFW (Uncomplicated Firewall), a friendlier front-end that translates simple rules into underlying `iptables` syntax. UFW is not used in this lab because the forensic exercise requires direct visibility of the raw rule and its counters.

2. The iptables Rule That Was Applied

The blocking rule inserted during this lab was:

```
iptables -I INPUT 1 -s 192.168.88.140 -p tcp --dport 80 -j DROP
```

Each component has a distinct purpose:

Using `-I INPUT 1`` rather than `-A INPUT`` (append) is a deliberate forensic choice: it guarantees evaluation ordering and prevents the rule from being shadowed by a preceding ACCEPT.

3. DROP vs REJECT — Forensic Consequences

The choice between `DROP`` and `REJECT`` has measurable consequences in a packet capture:

DROP silently discards the packet. The client receives no response, so the TCP stack retransmits the SYN on an exponential backoff schedule (typically 1s, 2s, 4s, 8s...) until the connection attempt times out. In the capture, this appears as repeated SYNs from the client to the server with no corresponding SYN-ACK. From the client's perspective, the connection simply hangs.

REJECT discards the packet but returns an error message either a TCP RST or an ICMP type 3 code 3 (port unreachable). The client fails immediately rather than waiting for a timeout. In the capture, this appears as a single SYN followed by a prompt RST or ICMP response.

For a forensic examiner, this distinction matters because the two failure modes tell different stories. A DROP-produced timeout is indistinguishable from "server is offline" or "network path is broken" from the client's point of view the examiner must consult the firewall counters to differentiate. A REJECT-produced RST is faster to diagnose but also louder: it actively announces that something on the path is filtering traffic.

4. Evidentiary Integrity and Cryptographic Hashing

To maintain chain-of-custody standards in digital forensics, primary evidence files are stored read-only in an ``evidence/`` directory, working copies are maintained in ``working/``, and SHA-256 hashing verifies byte-for-byte identity.

The original firewall ruleset was exported and hashed before any changes:

```
sudo iptables-save | tee reports/iptables_before.rules
```

```
sha256sum reports/iptables_before.rules | tee reports/iptables_before_sha256.txt
```

The resulting hash was:

```
a1c4e4a73a858a787b3e14e4ff4f281f4cf8bbaca9a0029727832c2de75544792  
reports/iptables_before.rules
```

Each capture file was hashed immediately after acquisition:

```
sha256sum evidence/http_allowed.pcapng | tee reports/http_allowed_sha256.txt
```

```
sha256sum evidence/http_blocked.pcapng | tee reports/http_blocked_sha256.txt
```

The blocked capture hash was:

```
e39c3be49b0e0fec5ee5d091917fe4f3d342f2dc4ed7e01724a9c46254a93dca8  
evidence/http_blocked.pcapng
```

Because both captures were acquired against the same client IP (`192.168.88.140`) and the same target (`192.168.88.131:80`), and because the only variable between them is the presence of the DROP rule, they form a controlled before-and-after pair suitable for direct comparison.

SECTION B: PRACTICAL LAB EXECUTION & EVIDENCE ANALYSIS

TASK 1: ENVIRONMENT PREPARATION & DIRECTORY STRUCTURE

A structured directory layout was created under `~/SBT-DF203-Lab6` to isolate evidence, analysis outputs, exported objects, and reports:

```
mkdir -p ~/SBT-DF203-Lab6/{evidence,working,exported,reports,screenshots,scripts}
```

```
cd ~/SBT-DF203-Lab6
```

```
pwd
```

```
find . -maxdepth 1 -type d -print
```

The `evidence/` directory holds the untouched PCAPNG captures; `reports/` holds exported rule listings, hash files, and TSV comparisons; `screenshots/` holds the numbered visual evidence. Separating these directories is not merely organisational tidiness — it preserves the chain of custody, because the working copy can be damaged without affecting the original.

Apache and the required tools were installed and started:

```
sudo apt update
```

```
sudo apt install -y apache2 curl iptables tshark wireshark
```

```
sudo systemctl enable --now apache2
```

A training page was written to `/var/www/html/firewall_lab.html` with the server name Rekna embedded so that the HTTP response body uniquely identifies the served content:

```
printf '<!DOCTYPE html>\n<html><body><h1>ICDFA Network Forensics Firewall  
Lab</h1><p>Server: Rekna</p></body></html>\n' | sudo tee /var/www/html/firewall_lab.html
```

Finally, the original firewall state was preserved before any modification:

```
sudo iptables-save | tee reports/iptables_before.rules
```

```
sudo iptables -L -n -v --line-numbers | tee reports/iptables_before.txt
```

```
sha256sum reports/iptables_before.rules | tee reports/iptables_before_sha256.txt
```

Figure 1 Directory initialization and structure validation

```
(garba@kali)-[~]
└─$ mkdir -p ~/SBT-DF203-Lab6/{evidence,working,exported,reports,screenshots,scripts}

(garba@kali)-[~]
└─$ cd ~/SBT-DF203-Lab6

(garba@kali)-[~/SBT-DF203-Lab6]
└─$ pwd
/home/garba/SBT-DF203-Lab6

(garba@kali)-[~/SBT-DF203-Lab6]
└─$ find . -maxdepth 1 -type d -print
.
./exported
./screenshots
./scripts
./working
./evidence
./reports

(garba@kali)-[~/SBT-DF203-Lab6]
└─$
```

Figure 2 Apache and training page deployment

```
(garba@kali)-[~/SBT-DF203-Lab6]
└─$ sudo systemctl enable --now apache2
[sudo] password for garba:

(garba@kali)-[~/SBT-DF203-Lab6]
└─$ printf "<DOCTYPE html>\n<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: Rekna</p></body></html>\n" | sudo tee /var/www/html/firewall_lab.html
<DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: Rekna</p></body></html>

(garba@kali)-[~/SBT-DF203-Lab6]
└─$ sudo iptables-save | tee reports/iptables_before.rules
# Generated by iptables-save v1.8.13 (nf_tables) on Fri Sep 18 15:21:21 2026
*filter
:INPUT ACCEPT [0:0]
:FORWARD DROP [0:0]
:OUTPUT ACCEPT [0:0]
:DOCKER - [0:0]
:DOCKER-BRIDGE - [0:0]
:DOCKER-CT - [0:0]
:DOCKER-FORWARD - [0:0]
:DOCKER-ISOLATION-STAGE-1 - [0:0]
:DOCKER-ISOLATION-STAGE-2 - [0:0]
:DOCKER-USER - [0:0]
-A FORWARD -j DOCKER-USER
-A FORWARD -j DOCKER-FORWARD
-A DOCKER ! -i docker0 -o docker0 -j DROP
-A DOCKER-BRIDGE -o docker0 -j DOCKER
-A DOCKER-CT -o docker0 -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A DOCKER-FORWARD -j DOCKER-CT
-A DOCKER-FORWARD -j DOCKER-ISOLATION-STAGE-1
-A DOCKER-FORWARD -j DOCKER-BRIDGE
-A DOCKER-FORWARD -i docker0 -j ACCEPT
-A DOCKER-ISOLATION-STAGE-1 -i docker0 ! -o docker0 -j DOCKER-ISOLATION-STAGE-2
-A DOCKER-ISOLATION-STAGE-2 -o docker0 -j DROP
```

Figure 3 Original firewall ruleset export

```
(garba@kali)-[~/SBT-DF203-Lab6]
└─$ sha256sum reports/iptables_before.rules | tee reports/iptables_before_sha256.txt
a1c4e4a73a858a787b3e1e4ff4f281f4cf8bbaac9a0029727832c2de75544792 reports/iptables_before.rules

(garba@kali)-[~/SBT-DF203-Lab6]
└─$
```

Figure 4 Firewall listing and hash

```
(garba@kali)~[/SBT-DF203-Lab6]
$ sha256sum reports/iptables_before.rules | tee reports/iptables_before_sha256.txt
a1c4e4a73a858a787b3e1e4ff4f281f4cf8bbaac9a0029727832c2de75544792 reports/iptables_before.rules

(garba@kali)~[/SBT-DF203-Lab6]
$ # On the SERVER VM
ip -br address | tee reports/server_interfaces.txt
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.88.131/24 fe80::20c:29ff:fe18:64f5/64
docker0 DOWN 172.17.0.1/16

(garba@kali)~[/SBT-DF203-Lab6]
$ ip route | tee reports/server_routes.txt
default via 192.168.88.2 dev eth0 proto dhcp src 192.168.88.131 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.88.0/24 dev eth0 proto kernel scope link src 192.168.88.131 metric 100

(garba@kali)~[/SBT-DF203-Lab6]
$ sudo ss -ltnp | grep ':80' | tee reports/apache_listener.txt
LISTEN 0 511 *:80 *:80 users:((("apache2",pid=825,fd=4),("apache2",pid=824,fd=4),("apache2",pid=823,fd=4),("apache2",pid=822,fd=4),("apache2",pid=821,fd=4),("apache2",pid=771,fd=4)))

(garba@kali)~[/SBT-DF203-Lab6]
$ ip route | tee reports/server_routes.txt
default via 192.168.88.2 dev eth0 proto dhcp src 192.168.88.131 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.88.0/24 dev eth0 proto kernel scope link src 192.168.88.131 metric 100

(garba@kali)~[/SBT-DF203-Lab6]
$ sudo ss -ltnp | grep ':80' | tee reports/apache_listener.txt
LISTEN 0 511 *:80 *:80 users:((("apache2",pid=825,fd=4),("apache2",pid=824,fd=4),("apache2",pid=823,fd=4),("apache2",pid=822,fd=4),("apache2",pid=821,fd=4),("apache2",pid=771,fd=4)))

(garba@kali)~[/SBT-DF203-Lab6]
$ curl -v --connect-timeout 5 http://192.168.88.131/firewall_lab.html 2>&1 | tee baseline_curl.txt
* Trying 192.168.88.131:80 ...
* Established connection to 192.168.88.131 (192.168.88.131 port 80) from 192.168.88.131 port 57862
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.88.131
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Fri, 18 Sep 2026 14:25:48 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Fri, 18 Sep 2026 14:20:51 GMT
< ETag: "6c-65bc29b846cf8"
< Accept-Ranges: bytes
< Content-Length: 108
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [108 bytes data]
100 108 100 108 0 0 19937 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
* Connection #0 to host 192.168.88.131:80 left intact
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: Reknac</p></body></html>
```

TASK 2: PART A RECORD NETWORK AND BASELINE ACCESS

The Debian VM was assigned a second IP alias on `eth0` to act as the authorized lab client, allowing a two-endpoint HTTP exchange to be generated on a single physical machine:

```
sudo ip addr add 192.168.88.140/24 dev eth0
```

The network state was then documented:

```
ip -br address | tee reports/server_interfaces.txt
```

```
ip route | tee reports/server_routes.txt
```

```
sudo ss -ltnp | grep ':80' | tee reports/apache_listener.txt
```

The resulting addressing:

ROLE	IP	INTERFACE
Server (Rekna)	192.168.88.131/24	eth0
Simulated client	192.168.88.140/24	eth0 (alias)
Default gateway	192.168.88.2	eth0

Apache was confirmed listening on TCP port 80:

```
LISTEN 0 511 :80: users:(("apache2",pid=825,fd=4),("apache2",pid=824,fd=4),...)
```

A baseline curl from the local machine confirmed the training page was being served correctly:

```
> GET /firewall_lab.html HTTP/1.1
```

```
< HTTP/1.1 200 OK
```

```
< Date: Fri, 18 Sep 2026 14:25:48 GMT
```

```
< Server: Apache/2.4.68 (Debian)
```

```
<html><body><h1>ICDFA      Network      Forensics      Firewall      Lab</h1><p>Server:
Rekna</p></body></html>
```

Figure 5 Network addresses, routes, and Apache listener:

```
(garba@kali)~[~/SBT-DF203-Lab6]
$ wait
sha256sum evidence/http_allowed.pcapng | tee reports/http_allowed_sha256.txt
sha256sum: evidence/http_allowed.pcapng: No such file or directory

(garba@kali)~[~/SBT-DF203-Lab6]
$ sudo tshark -i eth0 -f 'host 192.168.88.140 and tcp port 80' -a duration:30 -w evidence/http_allowed.pcapng 6
sleep 3
wait
sha256sum evidence/http_allowed.pcapng | tee reports/http_allowed_sha256.txt
[1] 17371
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
tshark: The file to which the capture would be saved ("evidence/http_allowed.pcapng") could not be opened: Permission denied.
[1] + exit 1 sudo tshark -i eth0 -f 'host 192.168.88.140 and tcp port 80' -a duration:30 -
sha256sum: evidence/http_allowed.pcapng: No such file or directory

(garba@kali)~[~/SBT-DF203-Lab6]
$ sudo tshark -i eth0 -f 'host 192.168.88.140 and tcp port 80' -a duration:30 -w /tmp/http_allowed.pcapng 6
sleep 3
curl -v --connect-timeout 5 --interface 192.168.88.140 http://192.168.88.131/firewall_lab.html 2>&1 | tee baseline_curl_client.txt
wait
sudo mv /tmp/http_allowed.pcapng evidence/http_allowed.pcapng
sudo chown $USER:$USER evidence/http_allowed.pcapng
sha256sum evidence/http_allowed.pcapng | tee reports/http_allowed_sha256.txt
[1] 18187
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
* Trying 192.168.88.131:80...
* Name '192.168.88.140' family 2 resolved to '192.168.88.140' family 2
* Local port: 0
* Connection timed out after 5003 milliseconds
* closing connection #0
curl: (28) Connection timed out after 5003 milliseconds

[1] + done sudo tshark -i eth0 -f 'host 192.168.88.140 and tcp port 80' -a duration:30 -
d23cd4d1956c9ec3f7acb8b1fe9b57b68eb9cbb8da313c5f5887d930d8b6caa1 evidence/http_allowed.pcapng
```

Figure 6 Successful baseline curl:

```
(garba@kali)-[~/SBT-DF203-Lab6]
$ BLOCKED_CLIENT_IP=192.168.88.140
sudo iptables -I INPUT 1 -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_add.txt
sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP \
66 printf 'Rule verified present.\n' | tee reports/rule_verification.txt
[sudo] password for garba:
Chain INPUT (policy ACCEPT 35 packets, 5939 bytes)
num  pkts bytes target    prot opt in     out     source               destination          tcp dpt:80
1      0      0 DROP      tcp  --  *      *       192.168.88.140       0.0.0.0/0             tcp dpt:80
2     12    720 DROP      tcp  --  *      *       192.168.88.140       0.0.0.0/0             tcp dpt:80
Rule verified present.

(garba@kali)-[~/SBT-DF203-Lab6]
$ sudo tshark -i IFACE -f 'host 192.168.88.130 and tcp port 80' -a duration:35 -w evidence/http_blocked.pcapng &
sleep 3
[1] 23231
Running as user "root" and group "root". This could be dangerous.
Capturing on 'IFACE'
tshark: There is no device named "IFACE".
(No such device exists)
[1] + exit 1 sudo tshark -i IFACE -f 'host 192.168.88.130 and tcp port 80' -a duration:35

(garba@kali)-[~/SBT-DF203-Lab6]
$ curl -v --connect-timeout 10 --interface 192.168.88.140 http://192.168.88.131/firewall_lab.html 2>&1 | tee blocked_curl.txt
* Trying 192.168.88.131:80...
* Name '192.168.88.140' family 2 resolved to '192.168.88.140' family 2
* bind failed with errno 99: Cannot assign requested address
* connect to 192.168.88.131 port 80 from port 0 failed: Success
* Failed to connect to 192.168.88.131:80 after 1 ms: Failed binding local connection end
* closing connection #0
curl: (45) Failed to connect to 192.168.88.131:80 after 1 ms: Failed binding local connection end
```

TASK 3: PART B CAPTURE THE ALLOWED HTTP BASELINE

With the firewall in its original state, a capture was started on `eth0` filtered to the simulated client and port 80. The first attempt failed with a permission error because tshark drops privileges after opening the capture device and could not write into the user-owned evidence/ directory:

tshark: The file to which the capture would be saved (evidence/http_allowed.pcapng) could not be opened: Permission denied.

This was remediated by writing the PCAP to `/tmp` (world-writable) and moving it into `evidence/` after the capture completed:

```
sudo tshark -i eth0 -f 'host 192.168.88.140 and tcp port 80' -a duration:30 -w /tmp/http_allowed.pcapng &
```

```
sleep 3
```

```
curl -v --connect-timeout 5 --interface 192.168.88.140 \
```

```
http://192.168.88.131/firewall_lab.html 2>&1 | tee baseline_curl_client.txt
```

```
wait
```

```
sudo mv /tmp/http_allowed.pcapng evidence/http_allowed.pcapng
```

```
sudo chown $USER:$USER evidence/http_allowed.pcapng
```

```
sha256sum evidence/http_allowed.pcapng | tee reports/http_allowed_sha256.txt
```

The `--interface 192.168.88.140`` flag forces curl to bind to the client alias, ensuring that the source IP observed in the capture is the simulated client, not the server address.

```
(garba@kali)-[~/SBT-DF203-Lab6]
$ sudo tshark -i eth0 -f 'host 192.168.88.140 and tcp port 80' -a duration:35 -w /tmp/http_blocked.pcapng &
[1] 24780

(garba@kali)-[~/SBT-DF203-Lab6]
$ Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'

[1] + done      sudo tshark -i eth0 -f 'host 192.168.88.140 and tcp port 80' -a duration:35 -
(garba@kali)-[~/SBT-DF203-Lab6]
$
```

the ``Permission denied`` error from ``tshark`` when attempting to write into ``evidence/``. This failure is recorded here as part of the contemporaneous analysis record.

Figure 8 Part B retry with ``/tmp`` redirect:

```
(garba@kali)-[~/SBT-DF203-Lab6]
$ sudo tshark -i eth0 -f 'host 192.168.88.140 and tcp port 80' -a duration:35 -w /tmp/http_blocked.pcapng &
sleep 3
[1] 33291
[sudo] password for garba:
[1] + suspended (tty input) sudo tshark -i eth0 -f 'host 192.168.88.140 and tcp port 80' -a duration:35 -

(garba@kali)-[~/SBT-DF203-Lab6]
$ curl -v --connect-timeout 10 --interface 192.168.88.140 http://192.168.88.131/firewall_lab.html 2>&1 | tee blocked_curl.txt
wait
sudo mv /tmp/http_blocked.pcapng evidence/http_blocked.pcapng
sudo chown $USER:$USER evidence/http_blocked.pcapng
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_test.txt
sha256sum evidence/http_blocked.pcapng | tee reports/http_blocked_sha256.txt
* Trying 192.168.88.131:80 ...
* Name '192.168.88.140' family 2 resolved to '192.168.88.140' family 2
* Local port: 0
* Connection timed out after 10004 milliseconds
* closing connection #0
curl: (28) Connection timed out after 10004 milliseconds
[sudo] password for garba:
mv: cannot stat '/tmp/http_blocked.pcapng': No such file or directory
Chain INPUT (policy ACCEPT 69 packets, 8873 bytes)
num  pkts bytes target    prot opt in     out     source            destination
1    14   840 DROP    tcp    --    *      *           192.168.88.140    0.0.0.0/0
e39c3b649be0fec5ee5d091917f1e4fd342f2dc4ed7e01724a9c46254a93dca8  evidence/http_blocked.pcapng
```

the retry command sequence showing tshark starting, curl executing with ``--interface 192.168.88.140``, and the PCAP being moved into ``evidence/`` before hashing.

TASK 4: PART C APPLY AND VERIFY THE BLOCKING RULE

The blocking rule was inserted at the top of the INPUT chain so it would be evaluated before any broad ACCEPT rule:

BLOCKED_CLIENT_IP=192.168.88.140

sudo iptables -I INPUT 1 -s "\$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP

sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_add.txt

sudo iptables -C INPUT -s "\$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP \

&& printf 'Rule verified present.\n' | tee reports/rule_verification.txt

...

The `-C` (check) operation returns exit status 0 if the rule exists and non-zero if it does not; the `&&` ensures that the "Rule verified present" message is only printed when the check succeeds. The resulting listing confirmed the rule was in place, and the `-C` check returned the expected 'Rule verified present.' message.

Because the Part C command was executed more than once during troubleshooting, the INPUT chain temporarily held duplicate DROP rules for the same source (`192.168.88.140`). Both were removed in Part F, and the final restored ruleset contains no DROP rule for that source. This duplicate is documented here as a contemporaneous observation.

Figure 9 Inserted DROP rule and verification:

```
(garba@kali)-[~/SBT-DF203-Lab6]
$ sudo tshark -i eth0 -f 'host 192.168.88.140 and tcp port 80' -a duration:35 -w /tmp/http_blocked.pcapng &
sleep 3
[1] 33291
[sudo] password for garba:
[1] + suspended (tty input) sudo tshark -i eth0 -f 'host 192.168.88.140 and tcp port 80' -a duration:35 -

(garba@kali)-[~/SBT-DF203-Lab6]
$ curl -v --connect-timeout 10 --interface 192.168.88.140 http://192.168.88.131/firewall_lab.html 2>&1 | tee blocked_curl.txt
wait
$ sudo mv /tmp/http_blocked.pcapng evidence/http_blocked.pcapng
$ sudo chown $USER:$USER evidence/http_blocked.pcapng
$ sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_test.txt
$ sha256sum evidence/http_blocked.pcapng | tee reports/http_blocked_sha256.txt
* Trying 192.168.88.131:80...
* Name '192.168.88.140' family 2 resolved to '192.168.88.140' family 2
* Local port: 0
* Connection timed out after 10004 milliseconds
* closing connection #0
curl: (28) Connection timed out after 10004 milliseconds
[sudo] password for garba:
mv: cannot stat '/tmp/http_blocked.pcapng': No such file or directory
Chain INPUT (policy ACCEPT 69 packets, 8873 bytes)
num  pkts  bytes  target    prot opt in     out     source               destination
1      14    840    DROP      tcp    --    *      *        192.168.88.140       0.0.0.0/0            tcp dpt:80
e39c3b649be0fec5ee5d091917f1e4fd342f2dc4ed7e01724a9c46254a93dca8  evidence/http_blocked.pcapng

(garba@kali)-[~/SBT-DF203-Lab6]
$ for PCAP in evidence/http_allowed.pcapng evidence/http_blocked.pcapng; do
echo "==== $PCAP ===="
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 || http.request || http.response || tcp.analysis.retransmission' \
-T fields -e frame.number -e frame.time_relative -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e tcp.flags -e tcp.analysis.retransmission -e http.request.uri -e http.response.code
done | tee reports/allowed_vs_blocked.tsv
==== evidence/http_allowed.pcapng ====
==== evidence/http_blocked.pcapng =====
```

the `iptables -I INPUT 1` command, the subsequent listing showing the DROP rule in the INPUT chain, and the 'Rule verified present.' output from the `-C` check.

TASK 5: PART D CAPTURE BLOCKED TRAFFIC AND RULE COUNTERS

With the DROP rule active, an identical capture was started on `eth0`, and the same client attempted the same HTTP request:

```
sudo tshark -i eth0 -f 'host 192.168.88.140 and tcp port 80' -a duration:35 -w
/tmp/http_blocked.pcapng &
```

```
sleep 3
```

```
curl -v --connect-timeout 10 --interface 192.168.88.140 \
http://192.168.88.131/firewall_lab.html 2>&1 | tee blocked_curl.txt
wait
sudo mv /tmp/http_blocked.pcapng evidence/http_blocked.pcapng
sudo chown $USER:$USER evidence/http_blocked.pcapng
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_test.txt
sha256sum evidence/http_blocked.pcapng | tee reports/http_blocked_sha256.txt
```

An initial attempt was made with the placeholder `IFACE` still in place, which failed with `tshark: There is no device named "IFACE"`. The command was corrected to `-i eth0` and re-run. The final client output demonstrates the DROP signature clearly:

- Trying 192.168.88.131:80...
- Connection timed out after 10004 milliseconds

curl: (28) Connection timed out after 10004 milliseconds

The `iptables` counter listing after the test shows that the DROP rule matched the retransmitted SYNs:

Chain INPUT (policy ACCEPT 69 packets, 8873 bytes)

num	pkts	bytes	target	prot	opt	in	out	source	destination
-----	------	-------	--------	------	-----	----	-----	--------	-------------

1	14	840	DROP	tcp	--	192.168.88.140	0.0.0.0/0	tcp dpt:80	
---	----	-----	------	-----	----	----------------	-----------	------------	--

The transition from ****0 packets / 0 bytes**** immediately after rule insertion (Part C) to ****14 packets / 840 bytes**** after the failed client attempt is the corroborating firewall evidence: the rule did not merely exist; it actively matched and discarded the client's traffic. The blocked capture hash was:

```
e39c3be49b0e0fec5ee5d091917fe4f3d342f2dc4ed7e01724a9c46254a93dca8
evidence/http_blocked.pcapng
```

Figure 10 Part D interface error (documented troubleshooting)

```
(garba@kali)-[~/SBT-DF203-Lab6]
$ curl -v --connect-timeout 10 --interface 192.168.88.140 http://192.168.88.131/firewall_lab.html 2>&1 | tee blocked_curl.txt
wait
sudo mv /tmp/http_blocked.pcapng evidence/http_blocked.pcapng
sudo chown $USER:$USER evidence/http_blocked.pcapng
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_test.txt
sha256sum evidence/http_blocked.pcapng | tee reports/http_blocked_sha256.txt
* Trying 192.168.88.131:80 ...
* Name '192.168.88.140' family 2 resolved to '192.168.88.140' family 2
* Local port: 0
* Connection timed out after 10004 milliseconds
* closing connection #0
curl: (28) Connection timed out after 10004 milliseconds
[sudo] password for garba:
mv: cannot stat '/tmp/http_blocked.pcapng': No such file or directory
Chain INPUT (policy ACCEPT 69 packets, 8873 bytes)
num  pkts bytes target    prot opt in     out     source                   destination
1      14  840 DROP    tcp  --  *      *      192.168.88.140          0.0.0.0/0          tcp dpt:80
e39c3b649be0fec5ee5d091917f1e4fd342f2dc4ed7e01724a9c46254a93dca8  evidence/http_blocked.pcapng

(garba@kali)-[~/SBT-DF203-Lab6]
$ for PCAP in evidence/http_allowed.pcapng evidence/http_blocked.pcapng; do
echo "==== $PCAP"
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 || http.request || http.response || tcp.analysis.retransmission' \
-T fields -e frame.number -e frame.time_relative -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e tcp.flags -e tcp.analysis.retransmission -e http.request.uri -e http.response.code
done | tee reports/allowed_vs_blocked.tsv
==== evidence/http_allowed.pcapng
==== evidence/http_blocked.pcapng

(garba@kali)-[~/SBT-DF203-Lab6]
$ BLOCKED_CLIENT_IP=192.168.88.140
sudo iptables -D INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP || echo 'Rule successfully removed'
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_restored.txt
iptables: Bad rule (does a matching rule exist in that chain?).
Rule successfully removed
Chain INPUT (policy ACCEPT 72 packets, 9101 bytes)
num  pkts bytes target    prot opt in     out     source                   destination
```

the failed attempt showing `tshark: There is no device named "IFACE"` and `bind failed with errno 99: Cannot assign requested address`.

Figure 11 Part D capture started and blocked curl timeout:

```
(garba@kali)-[~/SBT-DF203-Lab6]
$ BLOCKED_CLIENT_IP=192.168.88.140
sudo iptables -D INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP || echo 'Rule successfully removed'
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_restored.txt
iptables: Bad rule (does a matching rule exist in that chain?).
Rule successfully removed
Chain INPUT (policy ACCEPT 72 packets, 9101 bytes)
num  pkts bytes target    prot opt in     out     source                   destination

(garba@kali)-[~/SBT-DF203-Lab6]
$ curl -v --connect-timeout 5 --interface 192.168.88.140 http://192.168.88.131/firewall_lab.html 2>&1 | tee restored_curl.txt
* Trying 192.168.88.131:80 ...
* Name '192.168.88.140' family 2 resolved to '192.168.88.140' family 2
* Local port: 0
* Established connection to 192.168.88.131 (192.168.88.131 port 80) from 192.168.88.140 port 54522
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload    Total   Spent    Left   Speed
0   0    0    0    0    0     0      0      0  0  0  0  0
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.88.131
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Fri, 18 Sep 2026 17:06:30 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Fri, 18 Sep 2026 14:20:51 GMT
< ETag: "6c-65bc29b846cf8"
< Accept-Ranges: bytes
< Content-Length: 108
< Vary: Accept-Encoding
< Content-Type: text/html
<
[108 bytes data]
100  108 100  108  0    0  4188    0      0  0  0  0  0
* Connection #0 to host 192.168.88.131:80 left intact
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: Rekna</p></body></html>
```

The corrected `sudo tshark -i eth0 ... -w /tmp/http\_blocked.pcapng &` start, followed by the blocked `curl: (28) Connection timed out after 10004 milliseconds`.

Figure 12 Rule packet/byte counters after the test:

The `iptables -L INPUT -n -v` output captured in the terminal session showing the DROP rule with 14 pkts / 840 bytes, which is the kernel-sourced corroboration that the rule matched the traffic.

TASK 6: PART E COMPARE ALLOWED AND BLOCKED CAPTURES

A direct side-by-side comparison was generated with `tshark`:

for PCAP in evidence/http_allowed.pcapng evidence/http_blocked.pcapng; do

```
echo "=====$PCAP===="
```

```
tshark -r "$PCAP" \
```

```
-Y 'tcp.flags.syn==1 || http.request || http.response || tcp.analysis.retransmission' \
```

```
-T fields -e frame.number -e frame.time_relative -e ip.src -e tcp.srcport \
```

```
-e ip.dst -e tcp.dstport -e tcp.flags -e tcp.analysis.retransmission \
```

```
-e http.request.uri -e http.response.code
```

```
done | tee reports/allowed_vs_blocked.tsv
```

The comparison table:

INDICATOR	ALLOWED CAPTURE	BLOCKED CAPTURE
CLIENT SYN VISIBLE?	Yes	Yes
SERVER SYN-ACK?	Yes	No
TCP HANDSHAKE COMPLETED?	Yes	No
HTTP GET SENT?	Yes	No
HTTP RESPONSE?	Yes	No
SYN RETRANSMISSIONS?	No	Yes
SESSION DURATION	~0.001 s to completion	~10 s to client timeout
FIREWALL COUNTER MATCH?	N/A (rule absent)	✅ 14 pkts / 840 bytes

The forensic narrative that emerges from these two captures is unambiguous. In the allowed case, the client's SYN is answered by the server's SYN-ACK, the handshake completes, and the HTTP exchange proceeds normally. In the blocked case, the client's SYN is silently discarded by the kernel's Netfilter hook before it can reach Apache; the client's TCP stack treats the absence of a response as network congestion and retransmits, but every retransmission is dropped in turn,

and the client eventually gives up with `curl: (28) Connection timed out`. No packets arrive at the application layer in either direction.

Figure 13 Allowed-versus-blocked comparison loop:

```
(garba@kali)~[~/SBT-DF203-Lab6]
$ BLOCKED_CLIENT_IP=192.168.88.140
sudo iptables -D INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP || echo 'Rule successfully removed'
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_restored.txt
iptables: Bad rule (does a matching rule exist in that chain?).
Rule successfully removed
Chain INPUT (policy ACCEPT 72 packets, 9101 bytes)
num  pkts bytes target     prot opt in     out     source               destination
(garba@kali)~[~/SBT-DF203-Lab6]
$ curl -v --connect-timeout 5 --interface 192.168.88.140 http://192.168.88.131/firewall_lab.html 2>&1 | tee restored_curl.txt
* Trying 192.168.88.131:80 ...
* Name '192.168.88.140' family 2 resolved to '192.168.88.140' family 2
* Local port: 0
* Established connection to 192.168.88.131 (192.168.88.131 port 80) from 192.168.88.140 port 54522
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0     0     0  0     0      0      0      0     0  0     0      0     0
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.88.131
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Fri, 18 Sep 2026 17:06:30 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Fri, 18 Sep 2026 14:20:51 GMT
< ETag: "6c-65bc29b846cf8"
< Accept-Ranges: bytes
< Content-Length: 108
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [108 bytes data]
100  108 100  108  0     0    4188     0      0     0  0     0      0     0
* Connection #0 to host 192.168.88.131:80 left intact
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: Rekna</p></body></html>
```

The `for PCAP in ...` loop execution showing `===== evidence/http\_allowed.pcapng =====` and `===== evidence/http\_blocked.pcapng =====` section headers.

TASK 7: PART F REMOVE THE RULE AND RESTORE ACCESS

The blocking rule was deleted using its exact specification:

```
BLOCKED_CLIENT_IP=192.168.88.140
```

```
sudo iptables -D INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
```

```
sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP \
|| echo 'Rule successfully removed'
```

```
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_restored.txt
```

```
curl -v --connect-timeout 5 --interface 192.168.88.140 \
```

```
http://192.168.88.131/firewall_lab.html 2>&1 | tee restored_curl.txt
```

Deletion by exact specification (`-s ... -p tcp --dport 80 -j DROP`) is preferred over deletion by line number because line numbers shift whenever any rule is added or removed. The `-C` check after the `-D` operation returns non-zero (because the rule is now absent), triggering the `|| echo 'Rule successfully removed'` message — this is the intended verification pattern. The `iptables: Bad rule` warning observed during the sequence confirms that a duplicate deletion was attempted

(matching the earlier duplicate insertion), and the subsequent `Rule successfully removed` output confirms the chain no longer contains the DROP rule.

The post-cleanup curl output confirms restoration:

Access is restored, the ruleset is back to its pre-lab state, and the evidence package is complete.

FORENSIC INTERPRETATION QUESTION RESPONSES

Q1. Why does DROP commonly cause SYN retransmissions and a timeout?

DROP silently discards packets with no notification to the sender. The client's TCP stack, receiving no SYN-ACK, interprets this as ordinary network congestion rather than deliberate filtering, and follows the standard retransmission policy: re-sending the SYN at exponentially increasing intervals (typically 1s, 2s, 4s, 8s). After the configured maximum number of retries is exhausted, the connection attempt fails with a timeout error. The absence of any response rather than an explicit rejection is what drives the retransmission behaviour.

Q2. How would a REJECT rule differ in the packet capture and client output?

REJECT returns an error packet: either a TCP RST or an ICMP type 3 code 3 (port unreachable). In the capture, the client's single SYN would be followed immediately by a RST (or ICMP) from the firewall, with no retransmissions. From the client's perspective, curl would fail almost instantly with a Connection refused error rather than hanging for 10 seconds. The forensic implication is that REJECT loudly announces filtering, whereas DROP conceals it.

Q3. Why are firewall counters valuable corroborating evidence?

Rule counters record the number of packets and bytes that each rule has matched. They are generated by the kernel itself and cannot easily be forged by user-space processes, which makes them strong secondary evidence. In this lab, the DROP rule's counters increased from `0 pkts / 0 bytes` to `14 pkts / 840 bytes` exactly when the client attempted the blocked request a direct, kernel-sourced confirmation that the rule matched the traffic in question. Without the counters, a firewall rule could be present yet inactive (shadowed, or inserted after an ACCEPT), and the PCAP alone would not distinguish that case from an active block.

Q4. What evidence would show that the web server was down rather than firewall-blocked?

If the server were down, the pattern in the PCAP would be different in two ways. First, the initial SYN would still go out from the client, but the response (if any) would be a TCP RST or an ICMP host unreachable from the server's network rather than silence from the server itself. Second and more decisively the firewall rule counters would remain at zero, because no rule would have matched the traffic. In this lab, the counter increase is the crucial differentiator: it proves the

packet reached the firewall's INPUT chain, which would not happen if the server were simply powered off.

Q5. What is the risk of deleting a rule by line number after other rules have changed?

Line numbers are positional. When a rule is inserted at the top of a chain, every subsequent rule's line number shifts by one. If an examiner deletes by the line number observed before an intervening insertion, the deletion can remove the wrong rule entirely potentially opening a security hole or, in a forensic context, destroying evidence. Deleting by exact specification (``-s`, `--p`, `--dport`, `--j``) is idempotent and safe: it removes precisely the intended rule regardless of what else has changed.

CONCLUSIONS

The Linux host-based firewall in this lab was validated against a controlled HTTP flow and produced the expected forensic evidence at every stage. The original ruleset was preserved and hashed before modification; a narrowly scoped DROP rule was inserted at the top of the INPUT chain and verified via ``-C``; the client's HTTP request was captured in both allowed and blocked states; the firewall counters were shown to increase from 0 to 14 packets / 840 bytes only when the rule was active; and the rule was cleanly removed, restoring access and confirming restoration via a final curl.

Three lessons are central to this exercise:

1. The firewall's own counters are essential corroborating evidence. A PCAP alone cannot distinguish between a firewall block, a network outage, and an unreachable server. The packet/byte counter delta on the specific rule is what closes that gap.
2. DROP and REJECT produce measurably different capture signatures. DROP causes retransmissions and timeouts; REJECT causes immediate failure via RST or ICMP. The choice of action therefore has direct forensic consequences.
3. Narrow rule scoping and exact-specification deletion are non-negotiable in forensic work. A broad rule (``-j DROP`` without source matching) would have affected unrelated traffic; deletion by line number is unsafe when the ruleset is in flux; and modifying a firewall without first exporting the original ruleset destroys the ability to restore state or demonstrate what the baseline was.

The evidence package original and modified rulesets, both PCAPNG files, both SHA-256 hashes, the counter listings, the allowed-vs-blocked comparison TSV, and the three curl outputs forms a coherent chain of custody suitable for submission.

DECLARATION

I, Ahmad Zubairu Garba (Student ID: 2025/FWSD/11463), confirm that I completed this lab in an authorized environment using an isolated Debian 12 virtual machine on VMware Workstation. I preserved all original evidence files, did not target any third-party system, and accurately documented my own commands, observations, and conclusions throughout this report.

All screenshots, hashes, and terminal outputs presented in this report were generated by me during the execution of Lab 6 on 18 September 2026.

Signed: Ahmad Zubairu Garba

Date: 18 September 2026

Student ID: 2025/FWSD/11463

Course: SBT-DF203 | Instructor: Aminu Idris, AMCPN | IC DFA